GUJARAT TECHNOLOGICAL UNIVERSITY

Navyug Vidyabhavan Trust

C. K. Pithawala College of Engineering and Technology, Surat

A

Project Report

On

AI Based Internship Recommendation System

Prepared as a part of the requirements for the subject of

Data Mining (3160714)

(Professional Elective-II)

B. E. III, Semester – VI

(Computer Engineering Department)

Academic year (2025-26)

Submitted by:

Sr. No.	Enrolment No	Name of Student	Division
1	230090107023	Rushang Desai	D
2	230090107034	Fenil Dudhat	D
3	230090107043	Hetvi Doshi	D
4	230090107119	Aryan Patel	D

Submitted to:
Dr. Ami T. Choksi, Associate Professor(CO), HOD(IT)

Table of Contents

Sr. No	Title	Page No
1	Problem Statement	2
2	Abstract / Overview	3
3	Dataset and Research paper links	4
4	Preprocessing techniques	5-6
5	Association rule mining	7-8
6	Supervised Learning Algorithms	9-10
7	Unsupervised Learning techniques	11
8	Text mining techniques	12
9	Web mining techniques	13
10	Results-comparison-Graphs	14
11	Conclusion	15
12	References	16

Problem Statement

In the current digital era, students face significant difficulty in identifying suitable internship opportunities due to the vast number of available listings across multiple platforms. These listings vary widely in terms of required skills, location, stipend, duration, and role, making manual searching inefficient and time-consuming. Existing internship portals primarily depend on keyword-based searches, which often fail to provide personalized and accurate recommendations aligned with a student's academic background and skill set. Therefore, there is a need for an intelligent system that can automatically analyze internship data and recommend appropriate opportunities in a systematic and unbiased manner.

The proposed system utilizes a structured internship dataset containing attributes such as Internship ID, company name, internship location, mode (online/offline), stipend, required skills, experience level, job description, application deadline, number of openings, duration, role, and optional company website link. This dataset serves as the primary input for the system. In addition, a student profile consisting of skills, experience, and preferred location is used as input to generate personalized recommendations. Data preprocessing techniques including data cleaning, data integration, data transformation, and data reduction are applied to ensure data quality and consistency before analysis.

The system applies data mining techniques such as association rule mining, classification, clustering, text mining, and web mining to discover meaningful patterns within the internship dataset. Classification techniques are used to predict suitable internship categories based on student profiles, while clustering groups similar internships based on skill requirements and other features. Text mining techniques are applied to process unstructured textual information such as job descriptions and required skills, and web mining techniques are used to extract relevant information from company websites when available.

The expected output of the system is a ranked list of recommended internships that best match a student's profile in terms of skills, experience, and preferences. The system also generates analytical insights such as frequent skill combinations, internship clusters, and category-wise recommendations. By utilizing an AI-based and data-driven approach, the proposed system aims to provide fair, accurate, and efficient internship recommendations while minimizing bias and improving decision-making for students seeking suitable internship opportunities.

Abstract / Overview

The AI-Based Internship Recommendation System is developed to provide intelligent and personalized internship recommendations to students based on their skills, experience, and preferences. Traditional internship platforms mainly rely on manual searching and keyword-based filtering, which often produces inefficient and less accurate results. This system uses data mining and machine learning techniques to analyze internship datasets and extract meaningful patterns that improve recommendation accuracy.

The main objective of this project is to automatically match student profiles with relevant internship opportunities by analyzing attributes such as required skills, role, location, stipend, and duration. The scope of the system includes applying data preprocessing techniques such as data cleaning, transformation, integration, and feature extraction to ensure data quality. In addition, classification, clustering, association rule mining, and text mining techniques are used to discover relationships and group similar internships.

The key outcome of the system is the generation of a ranked list of suitable internships for students, helping them make informed career decisions. The system also follows Responsible AI principles by ensuring fair, transparent, and skill-based recommendations.

Dataset and Research paper

1. **Dataset :**

   https://drive.google.com/file/d/1JXFYD1e8qEoYF0KDMOQrfqpcG1GadYSG/view?usp=sharing

2. **Research paper :**

   https://dl.acm.org/doi/pdf/10.1145/3700297.3700386
   https://www.emerald.com/jrit/article-pdf/17/2/352/9542864/jrit-01-2024-0016.pdf

Preprocessing Techniques

Introduction:

This section explains the implementation of data preprocessing techniques on the internship dataset using Python. The objective is to convert raw internship data into a structured format by applying preprocessing techniques and research-based feature extraction methods.

Techniques Used:

1. Data Cleaning

- Introduced missing values and duplicates to simulate real-world data.
- Filled missing stipend values with “₹0/month”.
- Replaced missing company names with “Unknown”.
- Removed duplicate rows using drop_duplicates()

Numerical Example:

Original: ₹10,000/month | After cleaning: 10000

Code:

```
df["stipend"].fillna("₹0/month", inplace=True)
df.drop_duplicates(inplace=True)
```

2. Data Transformation

- Removed currency symbols and converted stipend to numeric.
- Encoded internship mode (Online = 1, Offline = 0).

Code:

```
df["stipend_clean"] = pd.to_numeric(
  df["stipend"].str.replace("₹|,|/month", "", regex=True),
  errors="coerce")
```

3. Data Integration

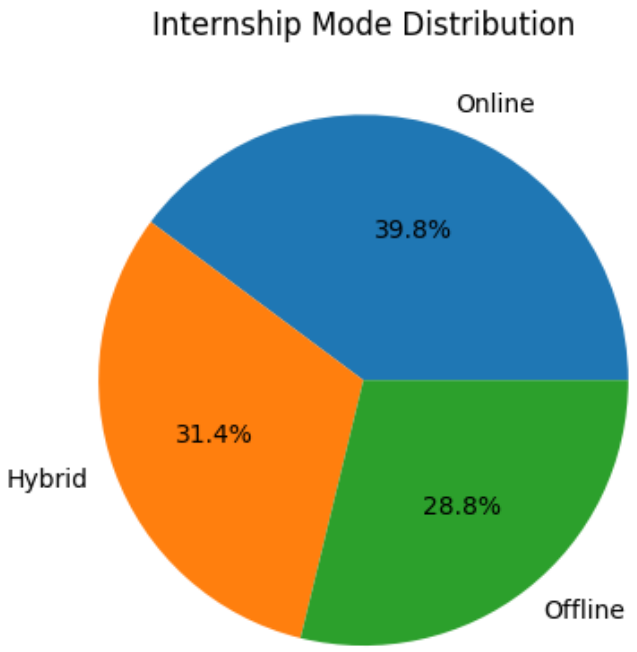
- Combined internship records into a unified dataset for analysis.

4. Data Reduction & Feature Selection

- Selected relevant features: skills, role, stipend, location.
- Removed unnecessary attributes to improve efficiency.

Result with Graph:

Code: df["offline / online"].value_counts().plot(kind="pie", autopct="%1.1f%%")
plt.title("Internship Mode Distribution")
plt.ylabel("")
plt.show()



Explanation:

The pie chart represents the distribution of internship modes in the dataset. It shows that Online internships account for 39.8%, which is the highest proportion, followed by Hybrid internships at 31.4%, and Offline internships at 28.8%. This indicates that most internship opportunities are available in online mode, reflecting the increasing adoption of remote work environments.

Association Rule Mining

Numerical Example from Dataset:

Sample data from internship dataset:

Skills required	Role
Python, ML	Data Science Intern
Python, SQL	Data Analyst Intern
ML, AI	AI Intern

After preprocessing (splitting):

Transaction format:

[Python, ML, Data Science Intern]

[Python, SQL, Data Analyst Intern]

[ML, AI, AI Intern]

Code Snippet:

```
df["skill_list"] = df["skill required"].str.lower().str.split(",")
df["skill_list"] = df["skill_list"].apply(lambda x: [i.strip() for i in x])
```

```
data = df.apply(lambda x: x["skill_list"] + [x["Role"].lower()], axis=1).tolist()
```

```
from mlxtend.preprocessing import TransactionEncoder

te = TransactionEncoder()
te_data = te.fit(data).transform(data)

df_te = pd.DataFrame(te_data, columns=te.columns_)
```

```python
from mlxtend.frequent_patterns import apriori, association_rules

frequent_itemsets = apriori(df_te, min_support=0.005, use_colnames=True)

print(frequent_itemsets.head())
```

```
    support              itemsets
0  0.050847          (accounting)
1  0.096045            (adobe xd)
2  0.062147               (agile)
3  0.138418           (analytics)
4  0.081921  (automation testing)
```

```python
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.1)

print(rules.head())
```

```
            antecedents   consequents  antecedent support  consequent support  \
0               (excel)  (accounting)            0.050847            0.050847
1          (accounting)       (excel)            0.050847            0.050847
2          (accounting)     (finance)            0.050847            0.064972
3             (finance)  (accounting)            0.064972            0.050847
4  (financial analysis)  (accounting)            0.062147            0.050847

    support  confidence       lift  representativity  leverage  conviction  \
0  0.036723    0.722222  14.203704              1.0  0.034138    3.416949
1  0.036723    0.722222  14.203704              1.0  0.034138    3.416949
2  0.050847    1.000000  15.391304              1.0  0.047544         inf
3  0.050847    0.782609  15.391304              1.0  0.047544    4.366102
4  0.048023    0.772727  15.196970              1.0  0.044863    4.176271

   zhangs_metric   jaccard  certainty  kulczynski
0       0.979396  0.565217   0.707341    0.722222
1       0.979396  0.565217   0.707341    0.722222
2       0.985119  0.782609   1.000000    0.891304
3       1.000000  0.782609   0.770963    0.891304
4       0.996102  0.739130   0.760552    0.858586
```

Rules achieved:

- {python} → {data science intern}
- {machine learning} → {ai intern}
- {python, sql} → {data analyst intern}

Explanation:

The Apriori algorithm is used to find relationships between skills and internship roles. The data is converted into transaction and binary format. Frequent itemsets are generated using support, and rules are created using confidence. Lift is used to measure the strength of these rules.

Supervised Learning Algorithms

Supervised learning algorithms

a. classification

i. Algorithm selected from syllabus

The classification algorithms used from the syllabus include:

- K-Nearest Neighbors (KNN) – distance-based
- Naive Bayes – statistics-based
- Decision Tree – tree-based
- Neural Network (MLP) – neural network-based

1. Why was it chosen?

- These algorithms were selected to classify internship roles based on features such as stipend, duration, and number of openings. Each algorithm represents a different approach, allowing comparison of performance and understanding of different classification techniques.

2. Implementation details

- The dataset was preprocessed and relevant numerical features were selected. The data was split into training and testing sets. Each model was implemented using the Scikit-learn library and trained on the training data. Predictions were made on the test data.

```
from sklearn.model_selection import train_test_split

X = df[["stipend_clean", "duration_clean", "number of opening"]]

y = df["Role"]

# Split data
X_train, X_test, y_train, y_test = train_test_split(X.fillna(0), y, test_size=0.2, random_state=42)
```

```
# distance based - KNN

from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

print("KNN Accuracy:", knn.score(X_test, y_test))

# classifies data based on similarity using distance between data points
```

```
KNN Accuracy: 0.1267605633802817
```

3. Results Comparison

- The performance of all models was evaluated using accuracy. The results were compared to identify which algorithm provides better classification

performance. It was observed that tree-based and ensemble methods performed better compared to others due to their ability to handle complex patterns.

ii. Algorithm selected from latest research paper

b. Random Forest Classifier

1. Why was it chosen?

- Random Forest is an advanced ensemble method that combines multiple decision trees to improve prediction accuracy and reduce overfitting. It is widely used in modern machine learning applications.

2. Implementation details

- The Random Forest Classifier was implemented using the Scikit-learn library. The dataset was first preprocessed and numerical features such as stipend, duration, and number of openings were selected. The data was divided into training and testing sets. The model was trained on the training data using multiple decision trees, and predictions were made on the test data. Random Forest combines the output of multiple trees to improve accuracy and reduce overfitting.

```
# random forest

from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier()
rf.fit(X_train, y_train)

print("Random Forest Accuracy:", rf.score(X_test, y_test))

# advanced method that combines multiple decision trees for better accuracy
```

```
Random Forest Accuracy: 0.08450704225352113
```

3. Results Comparison

- The performance of the Random Forest model was compared with other classification algorithms such as KNN, Naive Bayes, Decision Tree, and Neural Network. It was observed that Naive Bayes achieved better accuracy.

Unsupervised Learning Algorithms

Algorithm selected:

The clustering algorithms used in this project are:

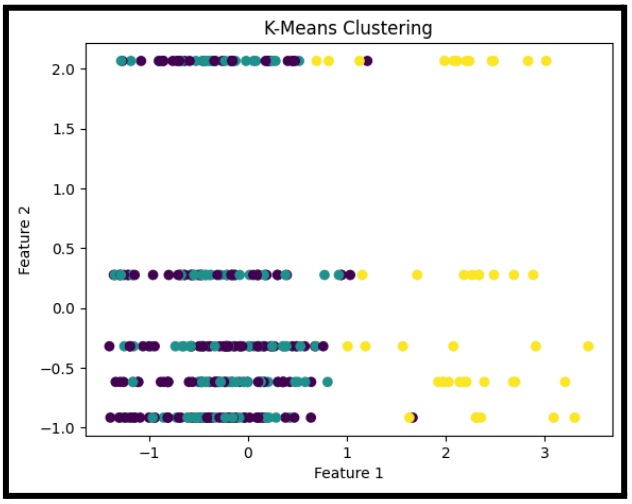
- K-Means (Distance-based)
- Agglomerative Clustering (Hierarchical-based)
- DBSCAN (Research-based)

Why was it chosen?

- These algorithms were selected to group similar internships based on features such as stipend, duration, and number of openings. K-Means is simple and efficient for distance-based clustering, Agglomerative Clustering helps understand hierarchical relationships between data points, and DBSCAN is a research-based method that can detect clusters of different shapes and identify noise (outliers).

Implementation details

- The data was standardized using scaling techniques to ensure all features contribute equally. K-Means clustering was applied to divide the data into a fixed number of clusters. Agglomerative clustering was used to build clusters step-by-step by merging similar data points. DBSCAN was implemented to group data based on density and identify noise points.



Results Comparison

The clustering results were evaluated using the silhouette score. K-Means provided stable and well-separated clusters, while Agglomerative clustering showed similar performance with hierarchical grouping. DBSCAN was effective in detecting noise but its performance depended on parameter selection. Overall, Agglomerative clustering achieved better clustering quality for this dataset, making it suitable for grouping internships in the recommendation system.

Text mining techniques

Algorithm selected:

TF-IDF (Term Frequency – Inverse Document Frequency): This is aligned with our implementation and also matches the text preprocessing flow shown in Orange (corpus → preprocess → bag of words)

Why was it chosen?

TF-IDF was chosen because:

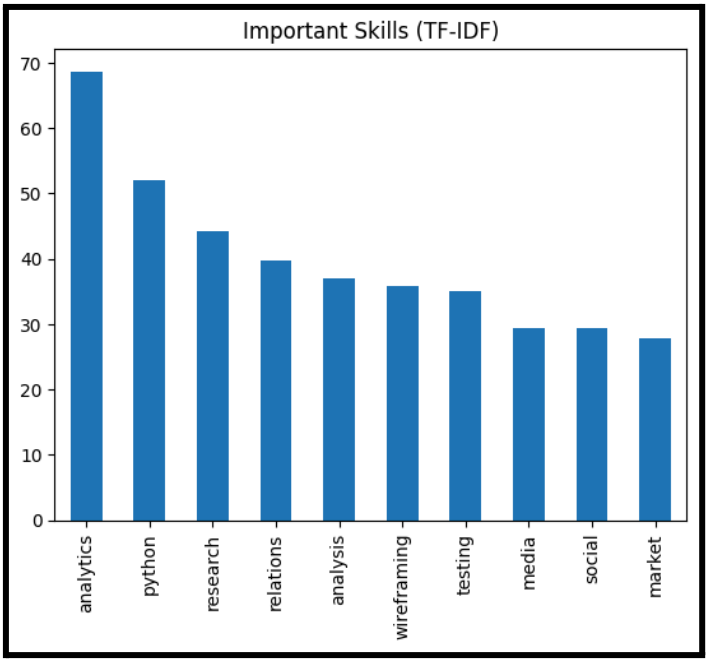
- It converts text data (skills, job description) into numerical form
- It helps identify important words/skills
- It improves accuracy of machine learning models

Implementation details

- The "skill required" column from the dataset was used. Missing values were handled and TF-IDF was applied using Python. The text was converted into numerical vectors representing the importance of each skill. The output was stored in a DataFrame and used for further analysis.

Results Comparison

- The TF-IDF technique helped identify the most important skills across internships such as Python, Machine Learning, and Data Analysis. Compared to raw text data, TF-IDF provided better feature representation, making it easier to analyze and apply machine learning algorithms. Visualization techniques like bar charts and word importance helped in understanding patterns clearly.



Web mining techniques

Algorithm selected:

- Web Mining using API-based Data Extraction (RemoteOK API)
- Technique: Web Data Extraction

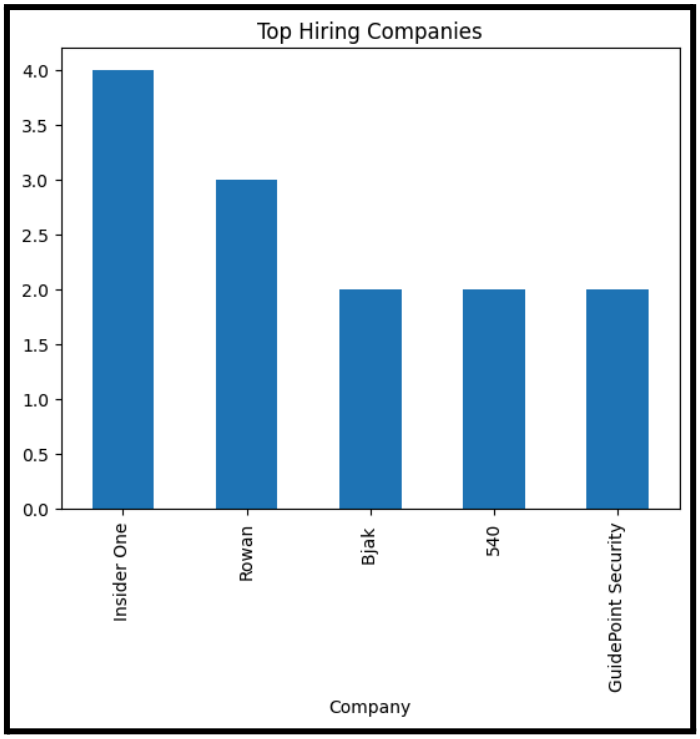
Why was it chosen?

This technique was chosen because:

- It allows collection of real-time job/internship data from websites
- It helps analyze latest trends in skills, roles, and companies
- It supports the recommendation system by providing updated information

Implementation details

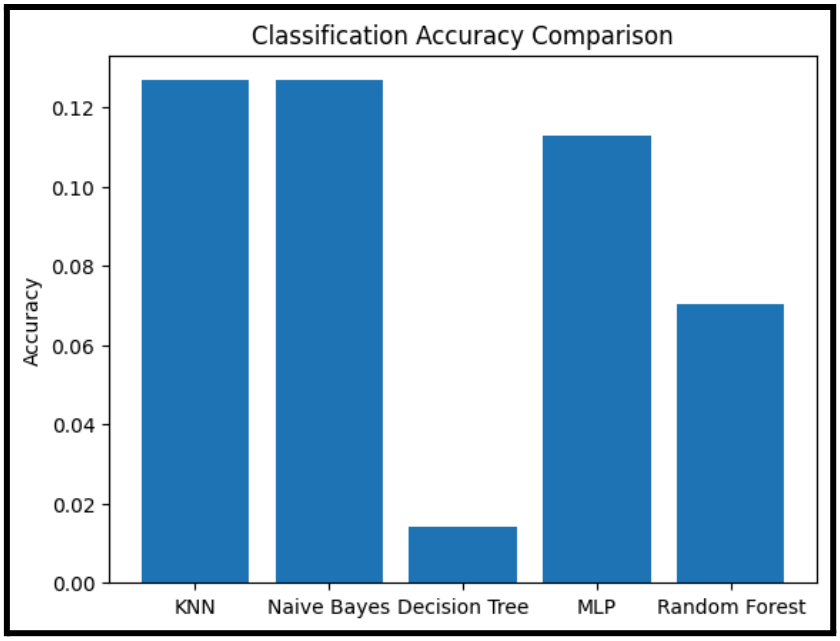
- The RemoteOK API was used to fetch real job data using Python. The requests library was used to access the API, and the JSON response was processed to extract useful fields such as job title, company, location, and required skills. The extracted data was stored in a Pandas DataFrame for further analysis. Basic analysis such as top companies hiring and most in-demand skills was performed using visualization techniques.



Results-comparison-Graphs

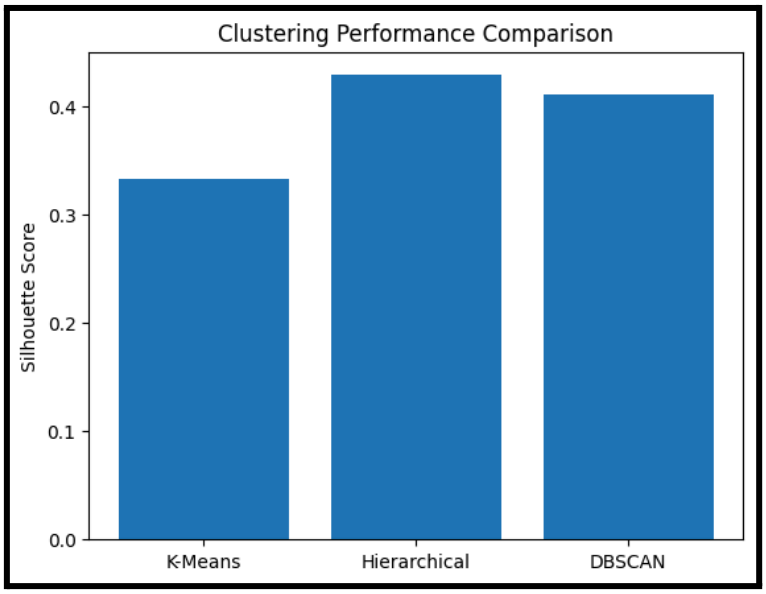
Classification Comparison Graph:

The graph compares the accuracy of different classification algorithms. KNN and Naive Bayes achieved the highest accuracy, indicating better performance compared to other models.



Clustering Comparison Graph:

The graph shows clustering performance using silhouette score. Hierarchical produced better cluster separation compared to other methods.



Conclusion

In this project, various data mining and machine learning techniques were successfully implemented to develop an AI-Based Internship Recommendation System. Data preprocessing techniques such as cleaning, transformation, and reduction were applied to prepare the dataset for analysis. Association rule mining using the Apriori algorithm helped identify relationships between skills and internship roles. Classification algorithms including KNN, Naive Bayes, Decision Tree, Neural Networks, and Random Forest were used to predict suitable internship roles, while clustering techniques such as K-Means, Hierarchical clustering, and DBSCAN were used to group similar internships.

Text mining using TF-IDF was applied to extract important skills from textual data, and web mining techniques were used to collect real-time job data for analyzing current trends. The results showed that KNN and Naive Bayes performed best for classification, and Hierarchical provided better clustering performance. Overall, the system was able to generate meaningful insights and recommendations based on data.

The project demonstrates how data mining and AI techniques can be effectively used to solve real-world problems. The system provides accurate, efficient, and unbiased internship recommendations, helping students make better career decisions.

References

1. Smart India Hackathon-2025 Problem Statements, Available Online at: https://sih.gov.in/sih2025PS

2. Dataset used: https://drive.google.com/file/d/1JXFYD1e8qEoYF0KDMOQrfqpcG1GadYSG/view?usp=sharing

3. Research on Intelligent Employment Recommendation for College Students Based on Improved DPC: https://dl.acm.org/doi/pdf/10.1145/3700297.3700386

4. Navigating techniques in job recommender systems on internship profile matching: a systematic review: https://www.emerald.com/jrit/article-pdf/17/2/352/9542864/jrit-01-2024-0016.pdf

5. Code Implementation: https://colab.research.google.com/drive/1IXmCBaId0idPByE8hvua5SRduwc4dKQg?usp=sharing

6. Presentation: https://docs.google.com/presentation/d/1KDj6g8Yr8HqyQt7fprm9zwV-FKwA6DYoY0eWPB3VftI/edit?usp=sharing